

Contents

Object Detection.....	1
Two-stage Detectors	1
One-stage Detectors	2
SSD (Single Shot Detector).....	3
RetinaNet	3
Semantic Segmentation.....	3
FCN (Fully Convolutional Network).....	4
SegNet	4
U-Net.....	4
DeepLab.....	4
Instance Segmentation	6
Mask R-CNN	6
Classification	9
Yolo (You Only Look Once)	11
U-Net	14

Object Detection

A method to find out what and where multiple objects are in an image

The model is trained on a dataset of pre-labeled images (including class labels and bounding box coordinates). Through this learning process, the model will automatically extract features to recognize objects and predict their locations in images.

Object detection algorithms are generally grouped into two main approaches: Two-stage Detectors and One-stage Detectors.

Two-stage Detectors

This approach processes the image in two separate steps: first proposing candidate regions that might contain an object (region proposal), then classifying each proposed region and refining its bounding box. This extra step makes two-stage detectors more accurate, but slower and less suitable for real-time use.

- R-CNN – the original version; slow because it processes each proposed region separately

- Fast R-CNN – improves speed by sharing computation across region proposals
- Faster R-CNN – uses a neural network (Region Proposal Network) to generate region proposals, making it faster still

How R-CNN works:

- 1. Uses an algorithm called Selective Search to scan the image and generate around 2,000 candidate regions that might contain an object.
- 2. Each region is cropped out separately and resized to a fixed size.
- 3. Every cropped region is passed through a CNN individually to extract features.
- 4. An SVM classifies each region, and a linear regressor refines its bounding box.

The main drawback is that the CNN has to run separately for each of the ~2,000 regions, which makes R-CNN very slow (tens of seconds per image).

Fast R-CNN improves this by running the CNN only once on the whole image to produce a shared feature map, then cropping the proposed regions from that feature map using a technique called RoI Pooling — avoiding the need to run the CNN repeatedly.

Faster R-CNN goes a step further by replacing Selective Search (a slow, CPU-based algorithm) with a Region Proposal Network (RPN) — a small neural network that learns to propose regions on its own, using anchor boxes similar to those in YOLO. This lets the entire pipeline run on the GPU, making it fast enough for near real-time use while keeping the two-stage accuracy advantage.

One-stage Detectors

This approach predicts an object's location and class in a single step, without a separate region-proposal stage. This makes one-stage detectors much faster and suitable for real-time applications, though sometimes slightly less accurate than two-stage detectors.

- YOLO (You Only Look Once) – divides the image into a grid and predicts boxes, confidence, and class directly for each cell
- SSD (Single Shot Detector) – predicts at multiple feature-map scales to detect both large and small objects
- RetinaNet – improves accuracy with a special loss function (Focal Loss) that addresses the imbalance between object regions and background

SSD (Single Shot Detector): like YOLO, SSD predicts boxes and classes in a single pass, but it makes predictions from multiple feature-map layers at different resolutions within the same network. Shallow layers (larger feature maps) are better at detecting small objects, while deeper layers (smaller feature maps) are better at detecting large objects — which helps SSD handle objects of very different sizes in the same image better than the original YOLO.

RetinaNet: one-stage detectors typically evaluate far more background regions than regions that contain an object, which creates a severe class imbalance during training. RetinaNet addresses this with Focal Loss, a modified version of Cross-Entropy Loss that automatically down-weights easy examples (background) and puts more emphasis on hard examples (real objects, unclear boundaries). This allows RetinaNet to reach accuracy close to two-stage detectors while keeping the speed of a one-stage detector.

Key Evaluation Concepts

Intersection over Union (IoU): measures how well a predicted bounding box overlaps with the true (ground-truth) box. It is calculated as the area of overlap between the two boxes divided by the area covered by both boxes combined. $\text{IoU} = 1$ means a perfect match, while $\text{IoU} = 0$ means the boxes do not overlap at all. An IoU of 0.5 or higher is commonly used as the threshold for a "correct" detection.

Confidence score: reflects how certain the model is that a predicted box both contains a real object and is positioned accurately. It is calculated as the probability that an object exists in the box, multiplied by the IoU between the predicted box and the true box. A low score in either factor — no real object present, or a poorly placed box — pulls the overall confidence down.

Applications

- Self-driving cars – detecting pedestrians, other vehicles, and traffic signs
- Security and surveillance cameras – detecting people or unusual objects in a scene
- Manufacturing – automated quality inspection of products
- Healthcare – detecting abnormalities in X-ray or MRI images

Semantic Segmentation

This means labeling every pixel in an image by its category, but it treats all objects of the same type as one group.

Example:

If a photo has 3 cats, all the pixels belonging to cats get colored the same color. The computer just knows this area is cat, but it doesn't know which pixel belongs to which specific cat.

Common Architectures

FCN (Fully Convolutional Network)

This is the model that really kicked off modern semantic segmentation. The key trick is swapping out the fully-connected layers at the end of a typical classification CNN (like VGG) for 1×1 convolutions instead. That one change means the network can accept an image of any size, and instead of producing a single label, it outputs a full spatial map. To bring that map back up to the original image size, FCN uses transposed convolution (sometimes called "deconvolution"). The catch is that shrinking the image down and then expanding it back out loses a lot of fine edge detail along the way, so FCN's output tends to look somewhat blurry around object boundaries.

SegNet

SegNet tackles that blurriness differently. Instead of learning the upsampling step from scratch, it remembers exactly where the maximum values were located during max-pooling in the Encoder, and reuses those same positions when expanding the image back out in the Decoder. This produces noticeably sharper object edges than FCN, without needing many extra parameters.

U-Net

What sets U-Net apart from FCN and SegNet is that it uses skip connections that directly concatenate Encoder feature maps into the Decoder, rather than just remembering pooling positions like SegNet. It was originally built for medical images and works well even with fairly small datasets.

DeepLab

DeepLab's biggest contribution is Atrous (Dilated) Convolution — a convolution with gaps inserted between the sampling points, which lets the network see a wider area of the image (a larger receptive field) without adding more parameters or shrinking the image further. On top of that, ASPP (Atrous Spatial Pyramid Pooling) runs several atrous convolutions with different dilation rates in parallel, helping the model recognize objects at very different scales within the same image. DeepLab is generally considered one of the more accurate segmentation architectures available today.

How It Works:

Most of these architectures follow roughly the same pattern: an Encoder (a CNN) extracts features while gradually shrinking the image down to a compressed, high-level representation; a Decoder then gradually expands that representation back out to the original size; and a final 1×1 convolution assigns a class probability to every pixel, producing an output image the same size as the input, with every pixel labeled.

Loss Functions

- Pixel-wise Cross-Entropy Loss – the most common choice; compares the predicted probability to the true label at each individual pixel
- Dice Loss – works better when the object of interest is much smaller than the background (a common class-imbalance issue), which is why it shows up so often in medical imaging

Many modern models combine both (sometimes called a "Combo Loss") to get the benefits of each.

Evaluation Metrics

mIoU (mean Intersection over Union) is the go-to metric for segmentation models. It calculates IoU separately for each class, then averages across all of them.

Pixel Accuracy is simpler — just the percentage of correctly classified pixels overall — but it can be misleading when the background dominates most of the image.

Applications

- Self-driving cars – segmenting lanes, sidewalks, pedestrians, and signs, since every pixel needs to be correctly identified to plan a safe path
- Satellite / remote sensing imagery – segmenting farmland, forests, urban areas, and bodies of water
- Healthcare – precisely outlining organs, tumors, or blood vessels in MRI or CT scans
- Photo editing – background removal, and features like Portrait Mode on smartphones

Instance Segmentation

This also labels by category, but it can tell each individual object apart, even if they're of the same type.

Example: With the same 3 cats, each cat gets a different color — Cat 1 is blue, Cat 2 is yellow, Cat 3 is red. The computer knows these are 3 separate cats, not just cats.

Common Architectures

Mask R-CNN

Mask R-CNN builds directly on top of Faster R-CNN. It keeps the same first stage — the Region Proposal Network (RPN) that proposes candidate regions — but adds a third output branch alongside the existing box and class branches: a small network that predicts a pixel-level mask for each detected object. So for every region Faster R-CNN would normally just draw a box around, Mask R-CNN also outlines the exact shape.

One important technical improvement Mask R-CNN introduced is RoIAlign, replacing the RoI Pooling used in Faster R-CNN. RoI Pooling rounds coordinates to fit a grid, which introduces small misalignments — barely noticeable for a bounding box, but significant when pixel-precise masks are needed. RoIAlign avoids that rounding, which is a big part of why Mask R-CNN's masks come out clean rather than jagged.

Pipeline

The full pipeline runs as follows: a backbone CNN (ResNet combined with a Feature Pyramid Network) extracts features at multiple scales; the Region Proposal Network (RPN) proposes candidate regions; RoIAlign extracts a fixed-size feature map for each proposed region; and finally three parallel branches process each region — a classification branch decides what class the object is, a box-regression branch refines the bounding box coordinates, and a mask branch predicts a pixel-level mask.

The Mask Branch

For each region of interest, the mask branch outputs a small $m \times m$ binary mask (typically 28×28) for every class the model can recognize — so with 80 possible classes, it predicts 80 separate small masks per region, and only the mask corresponding to the class chosen by the classification branch is kept. This deliberately decouples mask prediction from classification: the mask branch only has to decide "object or not object" pixel by pixel, without also worrying about which class it is, which keeps training more stable. The predicted $m \times m$ mask is then resized back up to match the actual bounding box before being placed onto the original image.

Loss Function

Mask R-CNN trains with a combined multi-task loss, since it solves three problems at once for every region proposal:

$$L = L_{\text{cls}} + L_{\text{box}} + L_{\text{mask}}$$

- L_{cls} – classification loss (log loss over object classes)
- L_{box} – bounding box regression loss (smooth L1 loss, same as Faster R-CNN)
- L_{mask} – average binary cross-entropy loss, computed only over the pixels of the ground-truth class's mask

Because L_{mask} only applies to the correct class's mask rather than all classes, the classification and mask branches do not interfere with each other during training — this separation is part of why Mask R-CNN performs well on both tasks simultaneously.

Backbone: ResNet + FPN

Mask R-CNN typically uses ResNet (50 or 101 layers) combined with a Feature Pyramid Network (FPN). FPN builds a pyramid of feature maps at several resolutions, connected top-down, allowing the model to detect and segment objects of very different sizes accurately: small objects are picked up by higher-resolution, shallower feature maps, while large objects are picked up by lower-resolution, deeper ones.

Evaluation Metrics

- Mask IoU – Intersection over Union computed between the predicted mask and the ground-truth mask, instead of between bounding boxes
- AP (Average Precision) at Mask IoU thresholds – following the COCO evaluation protocol, typically reported as AP50, AP75, and AP averaged over IoU thresholds from 0.5 to 0.95

YOLACT (You Only Look At Coefficients)

YOLACT takes a different approach, built for speed rather than maximum precision. Instead of generating a mask for each detected box individually, it does two things in parallel: it generates a small set of "prototype masks" for the whole image, and for each detected object it predicts a set of coefficients describing how to combine those prototypes into that object's specific mask. Combining prototypes with coefficients is a fast linear operation, which is what lets YOLACT run close to real time — something Mask R-CNN generally cannot do.

SOLO (Segmenting Objects by Locations)

SOLO skips the "detect a box first, then segment inside it" pipeline entirely. It divides the image into a grid, similar in spirit to YOLO, and each grid cell is made directly responsible for predicting the full mask of whichever object's center falls into it. Because there is no separate detection stage feeding into segmentation, this design tends to be simpler and faster than the two-stage Mask R-CNN approach.

How It Works:

Most instance segmentation models, Mask R-CNN in particular, follow this general flow: a backbone CNN extracts features from the input image; region proposals are generated (RPN) as candidate areas that might contain an object; then, for each proposed region, the model in parallel classifies the object, refines its bounding box, and predicts a pixel mask. The output for every detected object is therefore a class label, a bounding box, and a mask. The key difference from plain object detection is that third branch: the model is not just answering "what and where," it is also answering "which exact pixels."

Loss Function

Mask R-CNN trains with a combined multi-task loss, since it is solving three problems at once:

Total Loss = Classification Loss + Bounding Box Loss + Mask Loss

The mask branch itself is typically trained with pixel-wise binary cross-entropy, applied only within each object's own predicted region — the model only has to decide, pixel by pixel, whether it belongs to the object or not, separately for each detected instance.

Mask IoU: the same Intersection over Union concept used for bounding boxes, but computed on the predicted mask against the ground-truth mask instead of on rectangles.

AP (Average Precision) at various Mask IoU thresholds: commonly reported as AP50, AP75, or averaged across thresholds from 0.5 to 0.95 (the standard COCO evaluation protocol), giving a stricter and more informative measure than a single IoU cutoff.

Applications

- Robotics – picking individual items out of a cluttered bin, where the robot needs the exact outline of one object, not just its rough location
- Autonomous driving – separating overlapping pedestrians in a crowd, where a bounding box alone would not show where one person ends and the next begins
- Medical imaging – outlining individual cells or tumors when several are close together in the same scan
- Video editing / AR – isolating a specific person or object from the background for effects, background replacement, or object-aware filters

Classification

Classification is quite similar to Object Detection, except that Classification can only give a general overview of what the image is about — a single, overall label for the whole image.

Example: You give the model a photo of a cat. It just says cat. It doesn't know where the cat is, doesn't care if there's also a chair or a person in the background.

Common Architectures

LeNet-5

One of the earliest CNNs ever built for image classification, originally designed for handwritten digit recognition. It is tiny by today's standards, but it introduced the basic pattern almost every classifier since has followed: convolution layers to extract features, pooling layers to shrink things down, then fully-connected layers to make the final prediction.

AlexNet

AlexNet is generally credited with kicking off the deep learning boom in computer vision, after winning the ImageNet competition by a large margin in 2012. It is essentially a much deeper, much larger version of LeNet's idea, trained on GPUs at that scale for the first time. It also popularized ReLU activations and Dropout, both of which are still standard practice today.

VGG

VGG's contribution was showing that depth matters more than clever layer design — it stacks many small 3×3 convolution layers on top of each other instead of using larger filters. The architecture is very uniform and easy to understand, which made it popular as a backbone for other tasks, including the FCN and SSD models mentioned in earlier sections. Its main downside is size: VGG has a huge number of parameters, making it slow and memory-heavy.

ResNet (Residual Network)

ResNet solved a problem that was blocking progress at the time: simply stacking more and more layers was making networks harder to train, not better, because gradients would vanish on the way back through so many layers. ResNet's fix was the residual connection (or "skip connection") — letting each block learn only the difference from its input, and adding that input back in directly, rather than trying to learn a full transformation from scratch. This made it possible to train networks with over 100 layers, and ResNet remains one of the most widely used backbones today, including as the base for Faster R-CNN and Mask R-CNN.

EfficientNet

Instead of scaling a network by making it arbitrarily deeper, wider, or by feeding it larger images, EfficientNet scales all three dimensions together in a fixed, balanced ratio, called

compound scaling. This produces a family of models (EfficientNet-B0 through B7) that reach noticeably better accuracy for a given computational budget than earlier architectures.

How It Works:

Convolution and pooling layers are applied repeatedly, extracting features while progressively shrinking the spatial size and increasing the number of channels. The result is then flattened into a single feature vector, passed through one or more fully-connected layers, and finally through a softmax function that converts the output into a probability distribution over all classes. The output is one label for the whole image: the class with the highest probability.

Unlike detection or segmentation, classification never needs to know spatial detail — everything eventually collapses down into one vector describing what is in the image, not where.

Loss Function

Cross-Entropy Loss (specifically Categorical Cross-Entropy for multi-class problems) is the standard choice. It compares the predicted probability distribution from softmax against the true label, and penalizes the model more heavily the more confidently wrong it is.

$$\text{Loss} = - \sum (\text{true_label}_i \times \log(\text{predicted_probability}_i))$$

For example, if the true class is "cat" and the model predicts only 20% probability for cat, the loss will be high; if it predicts 95%, the loss will be low.

Evaluation

- Accuracy – the percentage of images classified correctly; simple, but can be misleading if the classes are imbalanced (e.g., 95% of images belonging to one class)
- Precision / Recall / F1-score – more informative when classes are imbalanced, since they separate false positives from false negatives
- Top-5 Accuracy – commonly used on large datasets like ImageNet (1,000 classes); counts a prediction as correct if the true label appears among the model's top 5 guesses, not just its single top guess

Applications

- Medical diagnosis – classifying an X-ray or skin lesion image as showing a disease or not
- Content moderation – automatically flagging inappropriate images on social platforms
- Quality control – classifying manufactured parts as defective or acceptable
- Everyday apps – plant and species identification apps, handwriting recognition, spam image filtering

Yolo (You Only Look Once)

Yolo handles the detection problem as a single regression problem: it divides the image into a grid, and each grid cell directly predicts the bounding box coordinates, confidence level, and class probability in just one scan of the network (one-shot), instead of having to scan the image multiple times like older methods.

Example:

I have an image that includes 1 car and 1 pedestrian

Yolo will divide the image into a grid, for example $4 \times 4 = 16$ cells

Car around cell 6,7

Pedestrian in cell 10

Yolo will determine which cell is responsible, in this case is cell 6 for the care and cell 10 for pedestrian

All the other cells from 1 to 16 that not responsible for any objects will be predict for nothing.

Next, cell 6 and cell 10 will each make a prediction, consisting of 3 things:

- Bounding box: the position and size of the object
- Confidence score: how sure the model is that an object exists there, and how accurate the box is
- Class probability: what the object is (car, person, etc.)

For example:

For Car: Yolo will predict cell around 6 (5-7), which one have the highest confidence score will be choose

For Pedestrian: Yolo will predict cell around 10 (10-11), which one have the highest confidence score will be choose

However, since the car spans across cells 6 and 7, cell 7 might also detect part of the same car and produce its own overlapping box for it. To avoid ending up with two boxes for the same object, YOLO applies **Non-Max Suppression (NMS)**: it compares how much the boxes overlap, keeps the box with a higher confidence score, and removes the other one as a duplicate.

Final result:

- Box 1: Car
- Box 2: Person

Architecture

A backbone network (originally Darknet, later CSPDarknet in newer versions) extracts features from the image, and the final layer outputs a tensor of size $S \times S \times (B \times 5 + C)$, where $S \times S$ is the number of grid cells, B is the number of predicted boxes per cell, and C is the number of classes.

For example, with the original YOLOv1 settings — $S = 7$, $B = 2$, $C = 20$ (PASCAL VOC dataset) — the output tensor size is:

$$7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

This means each of the 49 grid cells outputs a 30-value vector: 2 candidate boxes (5 values each) plus 20 class probabilities shared across the cell.

Key Techniques Behind the Algorithm

Intersection over Union (IoU)

IoU measures how well a predicted box overlaps with the true box:

$$\text{IoU} = \text{Area of Intersection} / \text{Area of Union}$$

IoU = 1 means a perfect match; IoU = 0 means no overlap at all. An IoU of 0.5 or higher is commonly used as the threshold for a "correct" detection.

Confidence Score

Each predicted box carries a confidence score reflecting both whether an object is really there and how accurate the box is:

$$\text{Confidence} = \text{Pr}(\text{Object}) \times \text{IoU}(\text{pred}, \text{truth})$$

Anchor Boxes

Later versions (YOLOv2 onward) assign several anchor boxes of different shapes to each cell, instead of just one. Each anchor has a pre-defined aspect ratio (tall-narrow, wide-short, etc.), so a cell can detect multiple overlapping objects of different shapes at once — a direct fix for the original YOLOv1 limitation, where a cell with two overlapping object centers could only predict one of them.

Non-Max Suppression (NMS)

Since neighboring cells often produce multiple overlapping boxes for the same object, YOLO removes the redundant ones: for each class, it keeps only the box with the highest confidence score and discards any other box that overlaps it beyond a set IoU threshold.

Multi-scale Prediction

Instead of predicting at only one grid resolution, newer versions predict at multiple feature-map scales simultaneously (e.g., 13×13 , 26×26 , 52×52) — coarse grids catch large objects, fine grids catch small ones, addressing another key weakness of the original design.

Loss Function

YOLO's loss combines several terms into one, since it optimizes location, confidence, and class prediction all at once:

Loss = Localization Loss + Confidence Loss + Classification Loss

- Localization Loss – squared error between predicted and true box coordinates (x, y, w, h), only computed for cells actually responsible for an object
- Confidence Loss – squared error between predicted and true confidence scores, computed separately for cells with and without an object
- Classification Loss – squared error (or cross-entropy in later versions) between predicted and true class probabilities

Applications

- Autonomous driving – real-time detection of pedestrians, vehicles, and traffic signs
- Video surveillance – live detection of people or objects of interest
- Robotics – real-time object detection for navigation and manipulation
- Retail / industrial automation – real-time product counting and defect detection on a moving line

U-Net

U-Net handles the segmentation problem as a pixel-wise classification problem: instead of predicting a single label or a bounding box for the whole object, it predicts a label for every single pixel in the image, producing an output image of the same size as the input, where each pixel is marked as belonging to an object or to the background.

U-Net has a symmetric U-shaped architecture made of two branches: the Encoder which gradually shrinks the image down while extracting features and the Decoder which gradually expands the image back to its original size while predicting a label for each pixel. A key feature is the skip connections, which directly link the Encoder to the Decoder at matching resolutions, helping to preserve spatial details (like object edges and exact positions) that would otherwise be lost during shrinking.

Example:

I have an MRI image that may contain a tumor.

U-Net's Encoder will repeatedly scan the image and shrink it down, step by step, learning general features like shape and texture at each stage.

At the deepest point, the image has been compressed into a small size, but it now holds rich, high-level information about what's in it.

Next, the Decoder starts expanding the image back up, step by step, gradually rebuilding it toward the original size.

At each expansion step, the Decoder borrows detailed information from the matching Encoder step through a skip connection, since fine details like exact edges tend to get lost during shrinking. Combining this borrowed detail with the general information already learned lets the Decoder rebuild the image both accurately and with sharp edges.

Once the image is expanded back to its original size, a final layer assigns each pixel a label — for example, "tumor" or "not tumor."

Final result:

A full-size output image where every pixel is labeled, clearly showing the exact location and shape of the tumor, not just a rough bounding box around it.

Architecture

U-Net has a symmetric U-shaped architecture made of two branches — an Encoder (contracting path) and a Decoder (expanding path) — connected by skip connections at matching resolutions.

Encoder (Contracting Path)

Repeats a block of two 3×3 convolutions (no padding, followed by ReLU), then a 2×2 max-pooling with stride 2, which halves the spatial size and doubles the number of feature channels. This block repeats 4 times in the original architecture, so by the deepest point, the image has been heavily compressed but holds rich, high-level feature information.

Decoder (Expanding Path)

Repeats a block of one 2×2 up-convolution (doubling the spatial size, halving the number of channels), then concatenates the result with the corresponding feature map from the Encoder via a skip connection, followed by two more 3×3 convolutions and ReLU. This also repeats 4 times, gradually rebuilding the image back to its original size. A final 1×1 convolution maps the result to the number of classes needed.

Feature-map Size Formula

The size of the feature map after each convolution or pooling operation follows:

$$\text{Output size} = \text{floor}((\text{Input size} - \text{Kernel size} + 2 \times \text{Padding}) / \text{Stride}) + 1$$

For example, with a 32×32 input, a 3×3 kernel, no padding, and stride 1:

$$\text{Output size} = \text{floor}((32 - 3 + 0) / 1) + 1 = 30$$

So a 3×3 convolution without padding shrinks each spatial dimension by 2 pixels — which is why U-Net's Encoder convolutions gradually reduce image size even before pooling kicks in.

Why Skip Connections Matter

As the image is compressed through the Encoder, fine spatial details — like exact object edges and positions — get progressively lost. Skip connections solve this by directly copying the Encoder's feature map at each resolution over to the matching stage in the Decoder, concatenating it with the upsampled features there. This lets the Decoder combine high-level context, learned deep in the network, with fine-grained detail preserved from earlier layers, producing segmentation masks with sharp, accurate boundaries instead of blurry ones.

Loss Functions

Pixel-wise Cross-Entropy Loss — the most common choice; compares the predicted class probability with the true label at every individual pixel.

Dice Loss — especially useful when the object of interest occupies a much smaller area than the background, a common issue in medical imaging (e.g., a small tumor in a large scan):

$$\text{Dice} = (2 \times |\text{Prediction} \cap \text{Ground Truth}|) / (|\text{Prediction}| + |\text{Ground Truth}|)$$

A Dice score of 1 means a perfect match between prediction and ground truth. Dice Loss is typically computed as $1 - \text{Dice}$. Unlike Cross-Entropy, which treats every pixel equally, Dice Loss focuses specifically on the overlap between predicted and true object regions — preventing the model from "cheating" by just predicting background everywhere, which is a real risk with severe class imbalance.

Evaluation Metrics

- Dice coefficient – same formula as above, used at evaluation time to measure overlap quality
- IoU (Jaccard Index) – Intersection over Union, applied at the pixel-mask level
- Pixel Accuracy – percentage of correctly classified pixels; can be misleading with imbalanced classes

Why U-Net Works Well with Limited Data

Unlike many deep learning models that need large labeled datasets, U-Net was specifically designed to perform well with relatively few training images — a critical requirement in medical imaging, where labeled scans are expensive and time-consuming to produce. Heavy use of data augmentation (elastic deformations, rotations) during training, combined with the skip-connection design, helps the model generalize from a small number of examples.

Applications

- Medical imaging – the original use case: segmenting tumors, organs, and cell structures in MRI, CT, and microscopy images
- Satellite imagery – segmenting roads, buildings, and vegetation
- Autonomous driving – pixel-level road and lane segmentation
- Industrial inspection – pinpointing exact defect regions on manufactured parts